CSC1031:OOP

Misha

Generics in Java

What is Generic?

A feature in Java that allows classes, interfaces, and methods to operate on any data type without sacrificing type safety.

Generics enable **parameterized types**, meaning you can write a class or method where the **type is a placeholder** (like T, E, K, V), and it will be specified later when the code is used.

```java
class Box<T> {
    private T value;

    public void set(T value) {
    this.value = value;
    }

    public T get() {
    return value;
    }
}
```

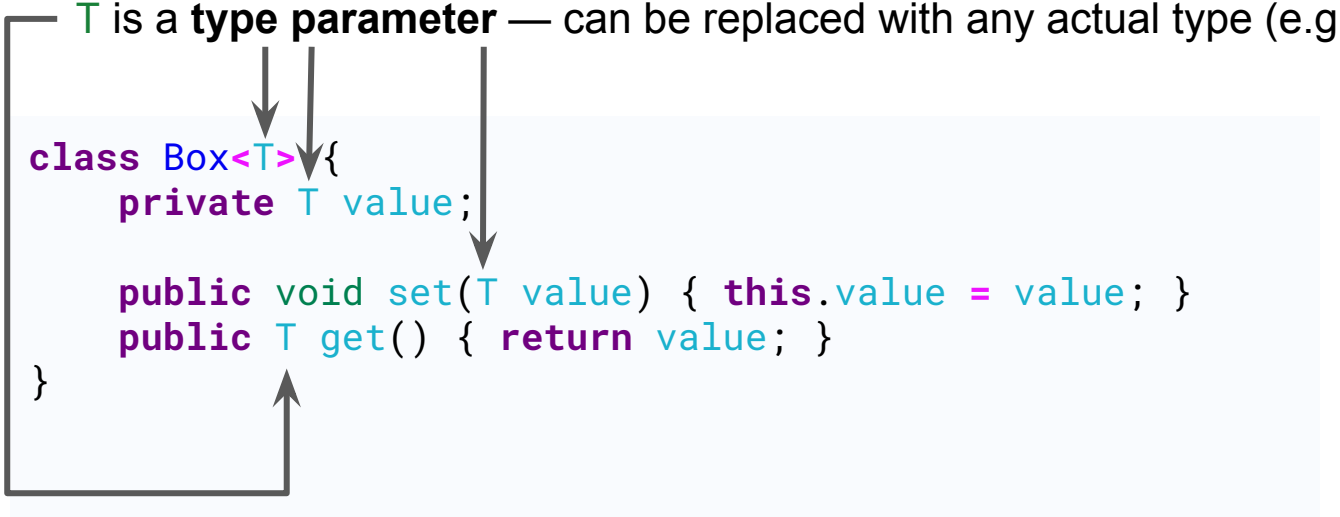
```java
List<String> list = new ArrayList<>();
```

```java
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
String s = stringBox.get(); // No casting needed
```

Generics = **Write once, use with any type** (String, Integer, custom classes, etc.)

Syntax of Generics

T is a **type parameter** — can be replaced with any actual type (e.g., `String`, `Integer`).



```java
class Box<T> {
    private T value;

    public void set(T value) { this.value = value; }
    public T get() { return value; }
}
```

```java
Box<String> stringBox = new Box<>();
stringBox.set("Janet");
```

Common Type Parameters

Symbol	Meaning	Where It's Used
`T`	**Type**	Generic classes and methods (`Box<T>`)
`E`	**Element**	Used in **collections** (`List<E>`)
`K`	**Key**	Used in **maps** (`Map<K, V>`)
`V`	**Value**	Used in **maps** (`Map<K, V>`)

Can I Use Any Letter in Generics?

Technically, **Yes.**

You can use **any valid identifier** (not just a single letter) as your type parameter in generics.

```
class Box<Banana> {
    private Banana item;
    public void set(Banana item) { this.item = item; }
    public Banana get() { return item; }
}
```

This is valid Java — but...

But Please Don’t

By convention, we use:

- **T** for Type
- **E** for Element
- **K, V** for Key and Value
- **N** for Number

These are well understood and expected by other developers.

Generic Methods

A generic method defines its own type parameter (like <T>) that can be used within its parameter list and return type.

<T> appears before the return type → this is required.

```java
public <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.println(element);
    }
}
```

```java
String[] words = {"Hello", "World"};
Integer[] numbers = {1, 2, 3};

printArray(words);    // prints each word
printArray(numbers);  // prints each number
```

Bounded Types

Sometimes you want your generic type to be limited to a specific type or its subclasses, e.g., only numbers.

```java
public <T extends Number> void printDouble(T value) {
    System.out.println(value.doubleValue());
}
```

This method accepts:

- `Integer`
- `Double`
- `Float`
- But **not** `String`, `Boolean`, etc.

Wildcards: ?

The **?** wildcard means “some unknown type.”

```
public void printAnything(List<?> list) {
    for (Object item : list) {
        System.out.println(item);
    }
}
```

Generic Method vs. Wildcard Parameter

Feature	Generic Method `<T>`	Wildcard Method `<?>`
Declares a new type?	✅ Yes (`<T>`)	❌ No
Can use type name in body?	✅ Yes (e.g., return `T` , use in args)	❌ No (only known as `Object`)
Can return the same type passed?	✅ Yes	❌ No (you don't know what it is)
Flexible for logic?	✅ More flexible	⚠️ Read-only / limited use

Declares its own **type parameter <T>**

Knows the exact type T during method execution

You can use T for other things too: return types, arguments, internal logic

Generic Method vs. Wildcard Parameter

Use **<T>** when:

- You want the method to know and work with a specific type
- You want to return or pass around that type

Use **<?>** when:

- You just want to read a list (or collection)
- You don’t care what’s inside, just need to loop over it

Wildcards - super and extend

"? extends T" → "Some unknown subtype of T (maybe T, maybe a child)"

"? super T" → "Some unknown supertype of T (maybe T, maybe a parent)"

extends = Java is protecting you from putting something too general into a more specific type

super = Java is protecting you from assuming a specific return type from a more general container

Wildcard Syntax	When to Use	Example Use Case
`<? extends T>`	**Read-only** — you consume values	Iterating or printing from a list
`<? super T>`	**Write-only** — you add values	Adding elements to a collection
`<?>` (unbounded)	**Read-only & general use** — unknown type	Logging, printing, size checks

Limitations of Generics

Type Erasure

At runtime, generic type information is erased

All List<String>, List<Integer>, etc. become just List

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();

System.out.println(a.getClass() == b.getClass()); // true
```

Limitations of Generics

No Primitives as Type Arguments

Java generics only work with reference types (objects), not primitives like int, double

```
List<int> nums = new ArrayList<>(); // ❌ Invalid

List<Integer> nums = new ArrayList<>(); // ✅ Use wrapper types
```

Limitations of Generics

Static Context Limitation

Static fields/methods can’t access type parameters, because T is only known per object

```
class MyClass<T> {
    // ❌ Illegal – static context can’t use instance type T
    static T value;
}
```

Best Practices with Generics

Don't Overcomplicate!

Generics should make code clearer, not harder to read

If you’re nesting multiple type parameters and confusing yourself — **refactor**

Antipatterns

What is an Antipattern?

An antipattern is a common coding solution that seems like a **good idea at first**, but leads to problems such as bugs, poor readability, or long-term maintenance issues.

- Often used with good intentions
- Looks like it works, but causes trouble over time
- Can make your code hard to test, debug, or extend

Antipattern #1 – God Object

A God Object is a single class that:

- Knows too much
- Does too much
- Controls everything

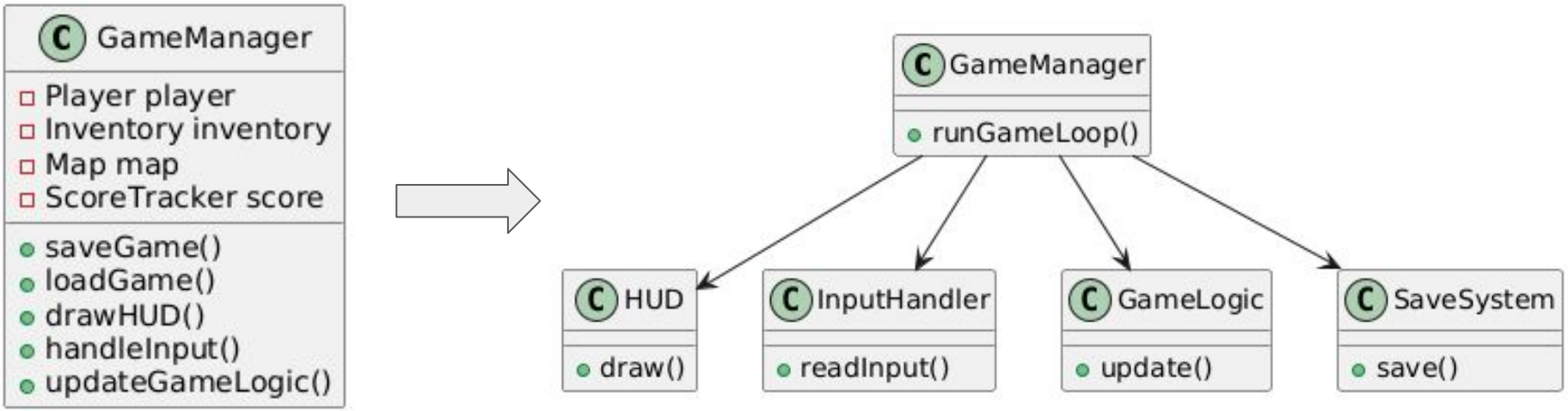
- ❏ Breaks Single Responsibility Principle (SRP)
- ❏ Hard to test, maintain, or understand
- ❏ Any change can create a ripple effect of bugs
- ❏ Other classes become dependent on it

Antipattern #1 – God Object - How to Fix It

Break into smaller, focused classes

Use composition and delegation

Apply design patterns like MVC, Mediator, or Strategy



Antipattern #2 – Spaghetti Code

Spaghetti code is a tangled, unstructured mess of logic where:

- Code has no clear flow or organization
- Methods do too many things
- Everything is interdependent

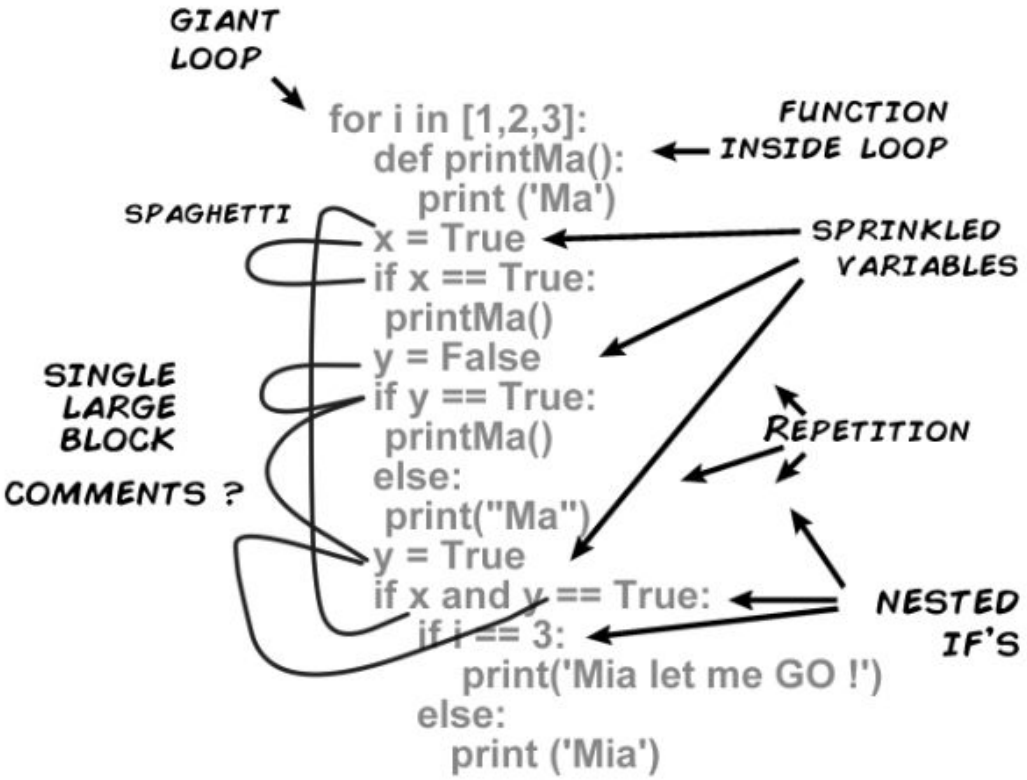
Think: “If I touch one line… everything breaks.”

Antipattern #2 – Spaghetti Code - Symptoms

- Massive main() methods
- No clear classes or methods
- Endless if / else if / switch / while loops
- Copy-pasted code blocks
- Impossible to follow or debug

Antipattern #2 – Spaghetti Code - How to Fix It

- Use **classes** and methods to separate logic
- Apply **polymorphism** instead of endless if/switch
- Follow the **Single Responsibility Principle**



Single Responsibility Principle (SRP)

“A class should have only one reason to change.”

It means each class should focus on one task, one job, one concern.

```
class ReportManager {
    void generateReport() { ... }
    void saveToFile() { ... }
    void sendEmail() { ... }
}
```

```
class ReportGenerator { void generate() { ... } }
class FileSaver { void save() { ... } }
class EmailSender { void send() { ... } }
```

Antipattern #3 – Inappropriate Inheritance

Inheritance used just to reuse code — not because the child class “is a” type of the parent.

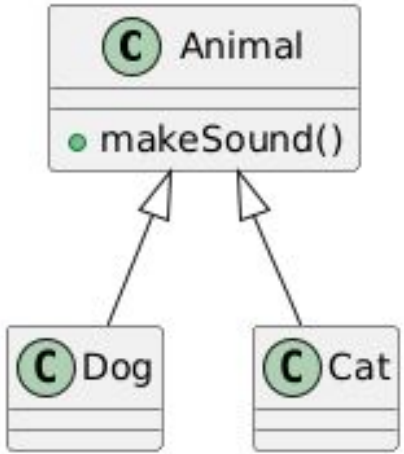
A `CoffeeMachine` is **not a** `Printer`, even if it shares a few methods.

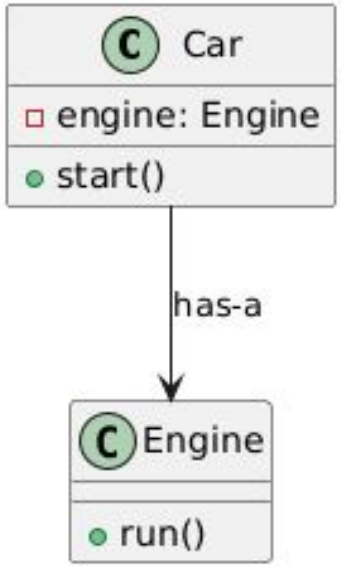
```
class Printer {
    void turnOn() { … }
    void turnOff() { … }
    void printDocument() { ... }
}

class CoffeeMachine extends Printer {
    void brewCoffee() { ... }
}
```

Antipattern #3 – Inappropriate Inheritance

Use inheritance only when there is a clear “is-a” relationship.

Otherwise, use composition.





Antipattern #4 – Switch Statement Overuse

Using switch (or giant if-else) blocks to control behavior instead of using **polymorphism**.

Often seen in procedural-style code inside OOP.

```
void processShape(Shape shape) {
    switch (shape.getType()) {
        case "circle":
            // calculate circle area
            break;
        case "square":
            // calculate square area
            break;
        case "triangle":
            // calculate triangle area
            break;
    }
}
```

Every time a new shape is added, you must **edit this method** — breaks **Open/Closed Principle**.

Antipattern #4 – Switch Statement Overuse

Let objects decide their own behavior

```
abstract class Shape {
    abstract double calculateArea();
}

class Circle extends Shape {
    double calculateArea() { return Math.PI * r * r; }
}

class Square extends Shape {
    double calculateArea() { return side * side; }
}
```

Antipattern #5 – Copy-Paste Programming

Repeating the same or slightly modified code in multiple places instead of reusing logic properly.

It often comes from:

- Time pressure ("I'll clean it later!")
- Fear of breaking something shared
- Lack of abstraction or understanding

Golden rule

If you’ve copied code twice — consider making it a method.

If you’ve copied it three times — you definitely should.

Antipattern #6 – Magic Numbers / Strings

Hard-coded numbers or strings in your code that:

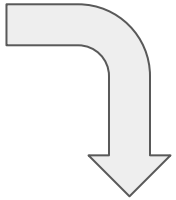
- Have no explanation
- Appear multiple times
- Seem to come from thin air

They make code confusing, fragile, and impossible to read without context.

Antipattern #6 – Magic Numbers / Strings - Example

```java
if (userAge >= 18) {
    System.out.println("Access granted");
}

if (discountCode.equals("X92B!")) {
    applyDiscount();
}
```

What does 18 mean?
What is "X92B!"?



```java
public static final int LEGAL_AGE = 18;
public static final String SPRING_DISCOUNT_CODE = "X92B!";

if (userAge >= LEGAL_AGE) {
    System.out.println("Access granted");
}

if (discountCode.equals(SPRING_DISCOUNT_CODE)) {
    applyDiscount();
}
```

- Self-documenting
- Easier to update
- Prevents accidental mistakes

Golden Rule

If a number or string has meaning, give it a name.